

DAT62-3

Feature Engineering and Model Selection

Albert School — 22 hours

Contents

Preface	2
1 Feature engineering strategy	3
2 High-cardinality encoding	4
2.1 Frequency encoding	4
2.2 Target encoding	4
2.3 Embedding-based encoding	5
3 Feature selection methods	6
3.1 Filter methods	6
3.2 Wrapper methods	7
3.3 Embedded methods	7
4 Leakage-free feature selection	8
4.1 The bias of selecting on all the data	8
4.2 Selection inside the cross-validation loop	8
5 Feature store design	9
6 Automated feature generation	10
7 Model selection under business constraints	11
7.1 A fair comparison across four families	11
7.2 The constraints that decide deployment	11
8 The written model selection report	12

Preface

By now you own the modeling cycle. **Intro to Machine Learning** (DAT32-1) taught you to split data, beat a baseline, train, evaluate on held-out data, regularize, and validate with k-fold cross-validation, all wired into a leak-proof scikit-learn pipeline. **Machine Learning I** (DAT32-3) added classification, the evaluation toolkit (precision, recall, F1, AUC-ROC), and the judgment to read them under asymmetric error costs. **Machine Learning II** (DAT42-1) gave you trees, random forests, and gradient boosting; hyperparameter search; feature scaling; a first pass at feature engineering (polynomial, interaction, target encoding, date decomposition); class-imbalance handling; and the habit of comparing model architectures in a written report. This book assumes all of it.

Two questions, though, were answered only in part. **Which features should the model see?** and **which of the competing models should I actually deploy?** DAT42-1 treated feature engineering as a grab-bag of transforms and model comparison as “fit three, pick the best AUC.” That is not enough for production. A feature computed slightly wrong leaks the answer and inflates your score until the day you deploy and it collapses. A model chosen on accuracy alone may be too slow for the latency budget, too opaque for the regulator, or too expensive to retrain weekly. This course sharpens both crafts into protocols you can defend.

The arc is two-act. The first act is **feature engineering**: how to enumerate candidate features from raw variables, how to encode high-cardinality categories without leaking, how to select the features that matter using filter, wrapper, and embedded methods, and — the crux — how to do that selection **inside** the cross-validation loop so it never sees the test set. We then make features durable and shareable through a **feature store**, and we learn to treat **automatically generated** features with healthy suspicion. The second act is **model selection**: a fair comparison across at least four algorithm families, scored not only on performance but on the constraints that decide real deployments — inference latency, interpretability, retraining cost — and written up in a report whose conclusion you can stand behind.

A word on method, unchanged from before: every idea arrives as a concrete example first and the general statement second. We keep one running problem throughout — **predicting which customers a bank should approve for a small loan**, where a wrong “approve” costs money, a wrong “decline” costs a customer, the latency budget is tight (a decision at the point of sale), and a regulator may demand an explanation. It is the kind of problem where every constraint in this course bites.

1 Feature engineering strategy

A raw dataset rarely arrives in the form a model learns from best. Feature engineering is the deliberate construction of the model’s inputs from the raw variables you are given. Before any encoding or selection, you need a **strategy**: a systematic way to go from raw columns to a list of candidate features, rather than inventing transforms ad hoc until the score stops moving.

Worked example — from three raw columns to a dozen candidates. The loan table has `birth_date`, `annual_income`, and `requested_amount`. A model fed these three raw numbers learns little. Thinking systematically: from `birth_date` derive `age` (a transformation); from the pair `requested_amount` and `annual_income` derive `debt_to_income = requested_amount / annual_income` (an interaction); bucket `age` into life stages; take `log(annual_income)` to tame its skew. Three raw columns have become a dozen candidate features — each one a **hypothesis** that this signal helps predict repayment.

Definition 1 A **feature engineering strategy** is the systematic enumeration of candidate features from a dataset’s raw variables. It proceeds in three moves:

1. **identify the raw variables** and their types (numeric, categorical, temporal, text, identifier) and what each one means in the business;
2. **propose candidate transformations** of single variables — scaling, log/power transforms to reduce skew, binning, date decomposition, encoding of categories;
3. **propose interaction terms** that combine two or more variables — ratios, products, differences — where domain knowledge suggests the combination carries signal the parts do not.

The output is a **candidate set**: a list of features to build and then test, each justified by a hypothesis about why it should help.

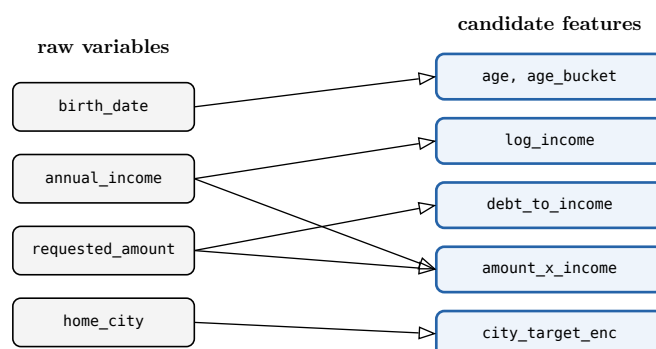


Figure 1: Feature engineering as a fan-out. Each raw variable spawns one or more candidate transformations, and pairs of variables spawn interaction terms (e.g. `requested_amount` and `annual_income` give `debt_to_income`). The strategy is the deliberate enumeration of this candidate set; selection (later chapters) then prunes it back down.

Common pitfall More features is not better. Every candidate you add is a chance to overfit, a column to compute and maintain in production, and noise that can drown the signal. The strategy **generates** candidates; you must still **select** among them (Chapters 3–4). Resist the urge to keep a feature just because you built it.

2 High-cardinality encoding

Categorical variables must become numbers before most models can use them. In DAT32-3 your default was **one-hot encoding**: one binary column per category. That works for `marital_status` (a handful of levels) and fails for `home_city` (10,000 levels), `merchant_id` (millions), or `zip_code`. One-hot encoding a high-cardinality column produces thousands of sparse columns — slow, memory-hungry, and starved of signal per column. This chapter covers three encodings built for high cardinality, and the leakage risk each one carries.

2.1 Frequency encoding

The simplest replacement throws away identity and keeps only how **common** a category is.

Definition 2 Frequency encoding (count encoding) replaces each category with the number of times it appears in the data (its count), or equivalently its relative frequency. A `home_city` of “Paris” seen 4,000 times becomes 4000 (or 0.04); a village seen twice becomes 2. One high-cardinality column collapses to a single numeric column.

Worked example — when frequency is the signal. For loan fraud, rare merchants are riskier than common ones. Frequency encoding turns `merchant_id` into a single column whose value **is** the rarity signal: tiny counts flag obscure merchants. The model never needs the merchant’s identity, only its commonness — and that is exactly what predicts the target here.

Frequency encoding leaks little, because a count does not directly involve the target — but it is blunt: two categories that happen to share a count become indistinguishable, even if one always repays and the other never does.

2.2 Target encoding

To carry real predictive signal, replace a category with what it tells you about the target. You met this in DAT42-1 as “target encoding”; here it is the workhorse for high cardinality, and we make its leakage discipline precise.

Definition 3 Target encoding (mean encoding) replaces a category with the mean of the target over the rows in that category — e.g. each city becomes the historical default rate of loans from that city. With **smoothing**, the estimate is shrunk toward the global mean for rare categories:

$$\text{enc}(c) = \frac{n_c \cdot \bar{y}_c + m \cdot \bar{y}}{n_c + m},$$

where $\bar{y}_c$ is the category mean, n_c its count, $\bar{y}$ the global mean, and m a smoothing strength. Rare categories (small n_c) lean on the global mean; common ones trust their own.

Common pitfall Target encoding is the **most powerful and most dangerous** of the three. Because the encoding is built from the target, computing a row’s encoding from a set that **includes that row’s own label** leaks the answer into the feature. The model then scores brilliantly in training and collapses in production. The danger peaks for rare categories: a city seen once gets encoded as that single customer’s own outcome — a perfect, useless predictor. **Always** target-encode out-of-fold (compute each row’s encoding from folds that exclude it) and smooth rare categories. We make this mechanical in Chapter 4.

2.3 Embedding-based encoding

Target and frequency encoding crush a category to a single number. When categories have rich structure — products that are “similar,” cities with related behavior — a single number is lossy. An **embedding** maps each category to a short vector of **learned** coordinates.

Definition 4 **Embedding-based encoding** represents each category as a dense vector of d learned real numbers (an **embedding**, typically d between 4 and 50). The vectors are learned so that categories with similar effects on the target end up close together in the embedding space. They are produced either as a by-product of training a neural network (an embedding layer) or by a dedicated representation-learning step.

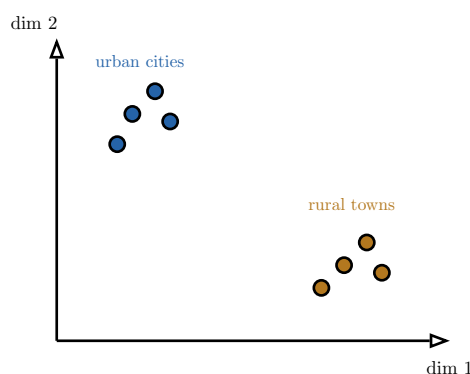


Figure 2: An embedding of `home_city` into two dimensions. Cities with similar repayment behavior land near each other, so the model can generalize across a **neighborhood** of cities rather than memorizing each one. A single target-encoded number could not express that two distant cities behave alike; a vector can.

Common pitfall Embeddings leak the same way target encoding does, only more subtly: they are learned **from the target**, so if the embedding is fit on the same rows you then train and evaluate on, target information seeps in. Learn embeddings on a separate split (or out-of-fold), and treat them as a fitted model step, never a one-shot transform of the full dataset. Embeddings also demand far more data than the other two encodings — with few rows per category there is nothing to learn, and frequency or smoothed target encoding wins.

Encoding	Output	Carries target signal?	Leakage risk
Frequency	1 column (count)	only indirectly (rarity)	low
Target / mean	1 column (mean y)	yes, directly	high — must go out-of-fold
Embedding	d columns (vector)	yes, learned	high + needs much data

Table 1: The three high-cardinality encodings. Frequency is safe but blunt; target encoding is informative but a leakage magnet; embeddings are richest but data-hungry and equally leak-prone. The right choice depends on cardinality, the number of rows per category, and the model family downstream.

3 Feature selection methods

The strategy of Chapter 1 deliberately over-produces candidates; encoding adds more columns still. Now we prune. **Feature selection** keeps the subset of features that helps and discards the rest — to reduce overfitting, speed up training and inference, cut the cost of computing and maintaining features in production, and make the model easier to interpret. Three families of methods exist, distinguished by **how much they involve the model**.

Worked example — three ways to ask whether a feature earns its place. Does `home_city_target_enc` deserve to stay? A **filter** asks: how correlated is it with default, ignoring every other feature and any model? A **wrapper** asks: does a model trained **with** it score better than the same model **without** it? An **embedded** method asks: when I train a Lasso or a tree, does it assign this feature a non-zero coefficient / any split importance? Same question, three escalating levels of model involvement — and cost.

3.1 Filter methods

Filters rank features by a statistic computed **independently of any model**, then keep the top ones. They are fast and model-agnostic, run once before training.

Definition 5 Filter methods score each feature by its statistical relationship with the target, independently of the model. Common scores:

- **Correlation** (Pearson) for numeric feature vs. numeric target — measures **linear** association, in $[-1, 1]$.
- **Mutual information** — measures **any** dependence (linear or not) between a feature and the target; zero iff they are statistically independent.

You rank features by the score and keep the top k , or those above a threshold.

Common pitfall Correlation sees only **linear** association. A feature with a perfect U-shaped relationship to the target (e.g. risk highest for both very low and very high incomes) has near-zero Pearson correlation and a filter would wrongly discard it. Mutual information catches such non-linear dependence. Conversely, filters judge each feature **in isolation**: they keep two perfectly redundant features that both correlate with the target, and they discard a feature that is useless alone but decisive in combination with another (the classic XOR trap). Use filters as a cheap first cut, not the final word.

3.2 Wrapper methods

Wrappers put the **model itself** in the loop: they search over feature subsets, training and scoring the model on each, and keep the subset that scores best.

Definition 6 Wrapper methods select features by repeatedly training the model on different feature subsets and using its validation performance as the score. **Recursive feature elimination (RFE)** is the standard: train the model on all features, rank them by the model's own importance (coefficients or tree importance), drop the weakest, and repeat on the survivors until a target number of features remains.

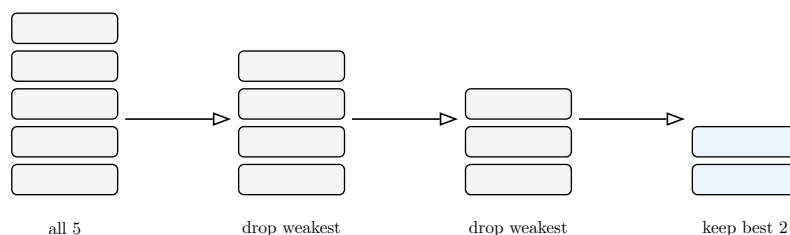


Figure 3: Recursive feature elimination. Train on the full set, rank features by the model's importance, drop the weakest, retrain on the rest, and repeat. RFE accounts for feature **interactions** the filter misses — but it trains the model many times, which is its cost.

Wrappers are powerful — they optimize the very metric you care about and account for interactions — but expensive, since they train the model many times, and they risk overfitting the selection to the validation folds if not cross-validated.

3.3 Embedded methods

Embedded methods get selection **for free** as a side effect of training a model that performs selection internally.

Definition 7 Embedded methods perform feature selection as part of model training. The two canonical cases:

- **Lasso (L1) regularization** drives the coefficients of unhelpful features exactly to zero (DAT32-1), so the surviving non-zero coefficients name the selected features.
- **Tree-based importance**: a random forest or gradient-boosting model ranks features by their total impurity reduction (DAT42-1); keep the high-importance features.

Selection and fitting happen in one training run, making embedded methods far cheaper than wrappers while still model-aware.

	Filter	Wrapper	Embedded
Uses the model?	no	yes, repeatedly	yes, once (during fit)
Cost	cheapest	most expensive	moderate
Sees interactions?	no	yes	partly
Examples	correlation, MI	RFE	Lasso, tree importance

Table 2: The three feature-selection families. Filters are fast and model-blind; wrappers are model-aware but costly; embedded methods are the practical middle ground, selecting as a by-product of one training run. A common workflow chains them: filter to a manageable set, then embedded or wrapper to finalize.

Common pitfall Tree-based importance inherits the bias you met in DAT42-1: it inflates the importance of high-cardinality features, which get more chances to split. When using embedded tree importance for **selection**, prefer permutation importance, or at least be suspicious of an ID-like column topping the ranking.

4 Leakage-free feature selection

This is the most important chapter in the book, and the one most teams get wrong. Feature selection **uses the target** — every method above scores features against y . If that scoring sees the data you later evaluate on, you have leaked, and your reported performance is a fantasy.

4.1 The bias of selecting on all the data

Worked example — selecting on the answer key. You have 1,000 features, most pure noise, and 200 customers. Before cross-validation, you compute each feature’s correlation with default **on all 200 rows** and keep the 20 most correlated. Then you cross-validate a model on those 20 and report a glowing AUC of 0.85. It is a mirage. With 1,000 noise features and 200 rows, **some** features correlate with the target **by chance**; selecting the top 20 on the full data handpicks exactly those lucky-noise features, and the cross-validation that follows is contaminated — every fold’s “held-out” rows already helped choose the features. Re-run with selection **inside** each fold and the AUC drops to 0.5: the features were noise all along.

Definition 8 Selection bias from feature selection is the optimistic bias introduced when features are chosen using the **same** data later used to estimate performance. Because selection peeks at the target across all rows, the held-out folds are no longer truly held out: information about them leaked into **which features exist**. The estimated performance is inflated, sometimes wildly, especially when the number of candidate features is large relative to the number of rows.

4.2 Selection inside the cross-validation loop

The cure is mechanical and absolute: feature selection is a **model step**, and like every model step (scaling, target encoding, imputation) it must be **fit on the training fold only**, then applied to the validation fold — never fit on the whole dataset.

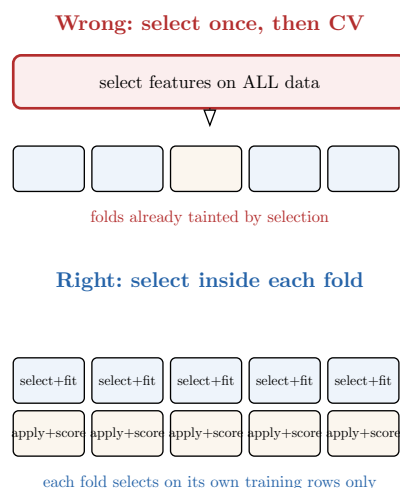


Figure 4: Leakage-free selection. **Wrong** (top): select features once on the whole dataset, then cross-validate — the held-out folds already shaped the feature set, so the score is inflated. **Right** (bottom): inside each fold, fit the selector on that fold’s **training** rows only, then apply it to the validation rows. The selection becomes part of what cross-validation honestly evaluates.

Definition 9 **Leakage-free feature selection** places the selection step **inside** the cross-validation loop, fitting it on each fold’s training portion and applying it to the held-out portion. In scikit-learn this means putting the selector (`SelectKBest`, `RFE`, `SelectFromModel`, the target encoder) **inside the Pipeline**, and passing the pipeline to `cross_val_score` / `GridSearchCV` — so the whole pipeline, selection included, is re-fit from scratch on every training fold. The resulting estimate is honest because no fold’s validation rows ever influenced its own feature set.

Common pitfall The single most common leakage bug in applied ML: calling `SelectKBest(...).fit(X, y)` or computing a target encoding on the **full X before** splitting, then cross-validating the model alone. The pipeline looks correct and the score looks great — and it is wrong. Rule: **anything that looks at y** — selection, target encoding, scaling parameters, imputation statistics, SMOTE — goes **inside** the pipeline, fit per fold. If you can compute it without the test set, it belongs in the pipeline.

5 Feature store design

A feature that works in your notebook is not yet a production asset. The same `debt_to_income` must be computed **identically** at training time and at serving time, reused across projects without each team reinventing (and subtly misdefining) it, and documented so its meaning and limits survive staff turnover. A **feature store** is the discipline (and often the tooling) that makes features durable, shareable, and trustworthy.

Definition 10 A **feature store** is a centralized, documented repository of features. For each feature it records at minimum:

- a **definition** — precisely what the feature measures, in business terms;
- a **source** — which raw tables/streams it derives from;
- **transformation logic** — the exact computation (so training and serving agree);

- **known limitations** — caveats, leakage hazards, populations where it is unreliable, and freshness/latency constraints.

It lets multiple models and teams reuse one authoritative definition of each feature instead of redefining it per project.

Worked example — one feature, one definition. Two teams both want “customer tenure.” Without a feature store, one computes it in months from `signup_date`, the other in days from `first_purchase_date`; the models disagree and nobody knows why. A feature store holds a single entry — `tenure_months`
`= floor((today - signup_date)/30)`, sourced from `accounts.signup_date`, limitation: “null for pre-2019 accounts; do not use for fraud where signup may be backdated” — and both teams consume the same definition. The store turns a feature from a notebook snippet into a governed asset.

Field	Example entry: <code>debt_to_income</code>
Definition	requested loan amount divided by annual income
Source	<code>loans.requested_amount</code> , <code>customers.annual_income</code>
Transformation	<code>requested_amount / annual_income</code> ; income floored at 1 to avoid div-by-zero
Known limitations	income self-reported and unverified; missing for 3% of applicants; meaningless for business accounts

Table 3: A single feature-store entry. Documenting definition, source, transformation, and limitations turns a feature into a reusable, auditable asset — and the “known limitations” field is the one teams skip and most regret, since it is where leakage hazards and unreliable populations get flagged.

6 Automated feature generation

Tools exist that **generate** features automatically — libraries that enumerate transformations and interactions across your tables and emit hundreds or thousands of candidate features with no manual effort. They are seductive and dangerous in equal measure. The skill the OP asks for is not to run them, but to **critically evaluate** what they produce.

Definition 11 Automated feature generation uses tools (e.g. deep feature synthesis in Featuretools, or AutoML feature builders) to mechanically construct many candidate features by applying and composing transformations across a dataset’s variables and relations. The output is a large candidate set produced without human hypotheses about which features should matter.

Common pitfall Automatically generated features must be **evaluated, never accepted blindly**. The hazards:

- **Leakage**: a generator that aggregates the target, or uses information unavailable at prediction time (e.g. “average of future transactions”), manufactures a perfect feature that cannot exist in production.

- **Meaninglessness:** most generated features are noise; keeping them inflates dimensionality and overfitting risk, and they carry no business interpretation you can defend to a regulator.
- **Unmaintainability:** a feature nobody understands cannot be debugged when it drifts.

Run automated generation as a **candidate source**, then subject every survivor to the leakage-free selection of Chapter 4 and a human sanity check: **can I explain why this feature should predict the target?** If not, it does not ship.

Worked example — the too-good feature. An automated generator produces `mean_balance_next_30d` and it tops every importance ranking with an AUC near 1.0. Suspicious, you read its definition: it averages the account balance over the 30 days **after** the loan decision — information that does not exist when the decision is made. It is pure leakage from the future. A human reading the feature’s **definition** caught what the impressive score never would. This is why generated features go through the feature store’s “definition” and “known limitations” fields, not straight into the model.

7 Model selection under business constraints

The second act. You have engineered and selected a clean feature set; now you must choose the model to deploy. DAT42-1 compared three architectures on accuracy. Real selection is harder on two counts: you compare across **more** algorithm families, and you score them on **more than performance** — the business constraints that decide what can actually run in production.

7.1 A fair comparison across four families

Definition 12 A **model comparison study** fits several algorithm families on the **same** engineered dataset, the **same** train/validation split, and the **same** cross-validation folds, scoring each by the **same** business-chosen metric — so the contest is fair and differences reflect the models, not the setup. The OP requires **at least four algorithm families**, for example:

1. a **linear/logistic** model (interpretable baseline);
2. a **tree ensemble — random forest** (robust, low-tuning);
3. a **gradient-boosting** model (often the accuracy leader on tabular data);
4. a **distance- or instance-based** model (e.g. k-NN) or an **SVM** or a **neural network**, to span a genuinely different inductive bias.

Common pitfall A comparison is only fair if everything except the model is held fixed. Two classic ways to rig it by accident: (1) tuning one model’s hyperparameters hard while leaving another at defaults; (2) applying scaling that one family needs and another is immune to **unequally**, or — worse — fitting any preprocessing on the full data so leakage helps some models more than others. Put preprocessing in the pipeline, give every family a comparable tuning budget, and evaluate all of them on identical folds.

7.2 The constraints that decide deployment

The best AUC does not win automatically. Three non-technical constraints, named in the OP, routinely overrule raw performance.

Definition 13 Business-constrained model selection scores candidate models not only on predictive performance but on:

- **Inference latency** — how long a single prediction takes. A point-of-sale loan decision needs an answer in milliseconds; a deep ensemble that takes 200 ms may be disqualified however accurate.
- **Interpretability** — whether a human can explain **why** the model decided as it did. Regulated decisions (credit, hiring, medicine) may **legally require** an explanation, ruling out opaque models regardless of accuracy.
- **Retraining cost** — the compute, data, and engineering needed to keep the model fresh as data drifts. A model that must be retrained nightly on huge data may cost more than its accuracy gain is worth.

The selected model is the one that best satisfies the **whole** set of constraints, not the one with the top score.

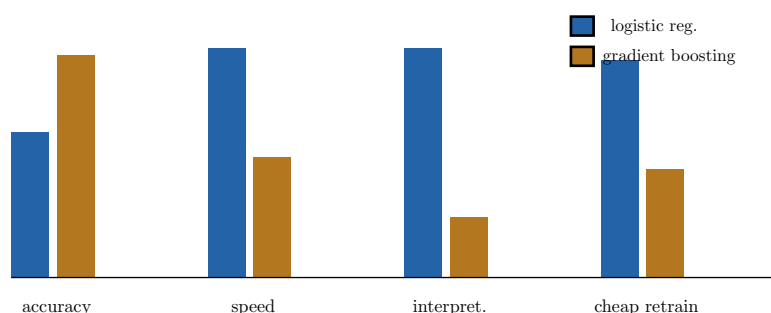


Figure 5: Two models across four axes. The booster (orange) wins on accuracy but loses on speed, interpretability, and retraining cost; logistic regression (blue) is the mirror image. There is no universally best model — only the best model **for this constraint set**. Naming the axes that matter, and weighting them, is the selection decision.

Worked example — when the lower AUC ships. On the loan problem the gradient booster scores AUC 0.91, logistic regression 0.86. On accuracy the booster wins. But the decision must be returned at the point of sale in under 50 ms (the booster’s ensemble takes 120 ms), and the banking regulator requires every declined applicant to receive a specific reason (the booster cannot produce one a human trusts; logistic regression’s signed coefficients can). Logistic regression **ships**, despite the lower AUC, because it is the only candidate that satisfies the latency and interpretability constraints. The booster’s five points of AUC are worthless if it cannot legally or physically run.

8 The written model selection report

The course converges on one deliverable: a **written report** that justifies the final model choice on **both** technical performance and non-technical constraints. The report is not paperwork; it is the artifact that makes a deployment defensible — to your manager, to a regulator, to the engineer who inherits the model in two years.

Definition 14 A model selection report documents and justifies the deployment decision. A complete report contains:

1. the **problem and metric** — the business task and the chosen evaluation metric, with justification for why that metric (recall the cost-asymmetry reasoning of DAT32-3);

2. the **feature set** — how features were engineered, encoded, and selected leakage-free, with the feature store as reference;
3. the **comparison** — at least four families on identical folds, with their scores;
4. the **constraint analysis** — each candidate’s latency, interpretability, and retraining cost;
5. the **decision** — the chosen model and an explicit argument weighing performance against constraints, including **why the runners-up were rejected**;
6. **limitations and monitoring** — known weaknesses and how the model will be watched for drift.

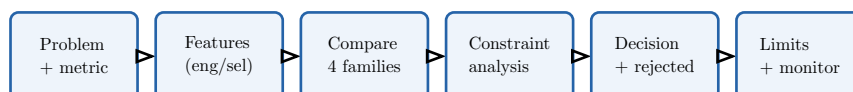


Figure 6: The model selection report as a chain of justified decisions. Each block is a section the reader can audit: the metric is justified by business cost, the features by leakage-free selection, the choice by performance **and** constraints, and the rejected models by an explicit argument. A report that omits the constraint analysis or the rejected-model reasoning is not defensible — it is just a leaderboard.

Common pitfall The weakest reports justify only the **winner** and only on **accuracy**. A defensible report does two things the weak one skips: it explains **why each rejected model was rejected** (so a reader cannot ask “but did you try X?”), and it weighs the **non-technical** constraints explicitly (so the choice survives a latency budget or a regulator’s audit). “It had the best AUC” is the beginning of the argument, never the end.

The arc of this course. Feature engineering begins with a **strategy** — enumerate raw variables, candidate transforms, and interactions — and continues with **high-cardinality encoding**: frequency (safe, blunt), target (powerful, a leakage magnet), embeddings (rich, data-hungry). You then **select** features with filter (cheap, model-blind), wrapper (RFE, costly, model-aware), or embedded (Lasso, tree importance) methods — always **inside the cross-validation loop**, because selecting on all the data leaks the target and inflates your score. Features become durable assets in a **feature store** (definition, source, transformation, limitations), and **automatically generated** features are evaluated, never trusted. The second act is **model selection**: a fair comparison across at least four algorithm families on identical folds, scored on performance **and** on the business constraints — inference latency, interpretability, retraining cost — that actually decide deployment. The whole course converges on a **written report** that justifies the final choice, rejects the alternatives explicitly, and weighs the numbers against the constraints. A model you cannot justify under real constraints is a model you have no business deploying.

Exercises

1. Take a structured dataset with at least one date column and one high-cardinality categorical column. Write a feature engineering **strategy**: list the raw variables and their types, propose candidate single-variable transformations, and propose at least three interaction terms, stating for each a hypothesis about why it should predict the target.

2. Encode a high-cardinality categorical variable three ways — frequency, target (smoothed), and an embedding (or a low-dimensional learned representation). Compare the validation performance and discuss the leakage risk of each. Demonstrate the target-encoding leakage failure by encoding in-fold and showing the inflated training score versus the honest out-of-fold score.
3. Select features using one method from each family: a filter (correlation or mutual information), a wrapper (recursive feature elimination), and an embedded method (Lasso or tree importance). Compare the selected subsets and explain a case where the filter keeps a redundant feature or discards a useful interacting one that the wrapper catches.
4. Construct a dataset with many noise features and few rows. Select features on the **full** data and cross-validate the model alone; then build a pipeline that selects **inside** each fold and cross-validate the pipeline. Report both AUCs and explain the gap as selection bias.
5. Design a feature store entry for three features from your project: for each, document the definition, source, transformation logic, and known limitations. Identify at least one feature whose “known limitations” field flags a leakage or data-availability hazard.
6. Run an automated feature generation tool on a dataset. From its output, find one feature that is leaking (uses future or target information) and one that is meaningless noise, and justify keeping or discarding each survivor by its definition, not its importance score.
7. Compare at least four algorithm families (e.g. logistic regression, random forest, gradient boosting, and k-NN or SVM) on the same engineered dataset and identical cross-validation folds, with preprocessing inside the pipeline. Produce a structured table of their scores under the business-chosen metric.
8. For the four models above, measure or estimate the inference latency, assess the interpretability, and estimate the retraining cost of each. Build a table scoring each model on accuracy **and** these three constraints.
9. Write a complete model selection report for your project: state the metric and why, describe the leakage-free feature pipeline, present the four-family comparison, analyze the constraints, name the chosen model, and justify the choice **and** the rejection of each runner-up on both technical and non-technical grounds.